

FPGA Implementation



Indian Institute of Technology
Hyderabad

Experiment : 07

EE2801: DSP Lab

Harshil Rathan Y

EE24BTECH11064

भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Contents

1	Experiment Objectives	2
2	FPGA Implementation and Board Bring-Up Procedure	2
2.1	Project Setup	2
2.2	Fixed-Point Representation	2
2.3	Coefficient and Data Generation	2
2.4	ROM IP Creation	3
2.5	RTL Integration	3
2.6	Timing Constraints	3
2.7	Pin Configuration	3
2.8	Programming the FPGA	3
2.9	SignalTap Analysis	3
2.10	Result Extraction	4
2.11	Testing Procedure	4
3	FIR Non Pipelined	4
4	FIR Pipelined	5
5	FFT Pipelined	6

1 Experiment Objectives

Implementing

- FIR filter, non-pipelined architecture
- FIR filter using a pipelined architecture
- 8 bit FFT using a radix-2

All modules use fixed-point arithmetic in Q(2,14) format, giving 17 total bits. This same width is used across the FIR coefficients, ROM data, board-level sample bus, and SignalTap observations so that the flow remains consistent for all three experiments.

2 FPGA Implementation and Board Bring-Up Procedure

This section describes the complete procedure to configure the FPGA board, generate input data, compile the design in Quartus Prime, and verify outputs using SignalTap.

2.1 Project Setup

- Create a project directory named `fir_single_mac`.
- Initialize a Quartus Prime project with the same name.
- Select FPGA device: **Cyclone V (5CSEBA6U23I7)**.
- Organize the project directory as follows:
 - `src/rtl` - RTL design files
 - `src/tb` - Testbench files
 - `data` - Input/output data files
 - `ip` - Generated IP cores
 - `scripts` - Python scripts

2.2 Fixed-Point Representation

All signals are represented in **Q(2,14)** format (17-bit signed fixed-point representation).

2.3 Coefficient and Data Generation

- A Python script (`coeff_gen.py`) is used to generate FIR filter coefficients as a Verilog module.
- Input signal data is generated using a Python script and stored as `.mif` files.
- The memory initialization file follows:

```
WIDTH=17;
```

```
DEPTH=1000;
```

```
ADDRESS_RADIX=UNS;
```

```
DATA_RADIX=HEX;
```

```
CONTENT BEGIN
```

```
0: data0
```

```
...
```

```
END;
```

- Signals are generated for frequencies: 900 Hz, 1100 Hz, and 2000 Hz.
- Sampling frequency is fixed at 100 kHz for correct operation.

2.4 ROM IP Creation

- Create a **1-Port ROM** using Quartus IP Catalog.
- Configuration:
 - Data width: 17 bits
 - Depth: 10000
 - Single clock operation
- Load generated .mif file as memory initialization.
- Enable *In-System Memory Content Editor*.

2.5 RTL Integration

- Implement the following modules:
 - FIR Filter (Non-pipelined / Pipelined)
 - Data Generator (ROM wrapper)
 - Top-level wrapper (de10_nano_top)
- The data generator feeds samples to the FIR filter based on ready/busy signals.
- Use (* keep *) attributes to prevent signal optimization.

2.6 Timing Constraints

- Create an SDC file specifying:


```
create_clock -name clk -period 20 [get_ports FPGA_CLK]
```
- This sets the clock frequency to 50 MHz.

2.7 Pin Configuration

Using Pin Planner:

- KEY[0] (Reset) → PIN_AH17
- FPGA_CLK → PIN_V11
- I/O standard: **3.3V LVTTL**

2.8 Programming the FPGA

- Connect DE10-Nano board via USB Blaster.
- Open Quartus Programmer.
- Click Auto Detect and select device 5CSEBA6.
- Load the generated .sof file.
- Start programming.
- Verify that the CONF_DONE LED is ON.

2.9 SignalTap Analysis

- Open SignalTap Logic Analyzer.
- Select clock signal from design.
- Add nodes:
 - Input signal (from data generator)

- Output signal (from FIR filter)
- Ready signal
- Group signals as buses (MSB to LSB order).
- Set:
 - Sample depth: 8K
 - Trigger: Rising edge of ready signal
- Compile and reprogram FPGA.
- Run SignalTap to capture waveform.

2.10 Result Extraction

- Convert captured signals to signed decimal format.
- Export waveform data as .csv.
- Use Python scripts to plot input and output signals.

2.11 Testing Procedure

- Load different input signals (900 Hz, 1100 Hz, 2000 Hz).
- Observe filtering behavior for:
 - Non-pipelined FIR
 - Pipelined FIR
 - Feedforward structure
- Capture and compare output responses.

3 FIR Non Pipelined

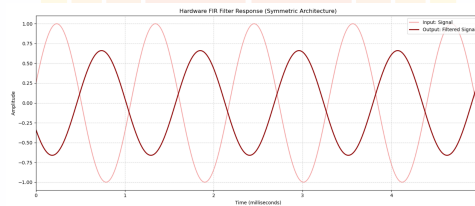


Fig. 0.1: 900Hz

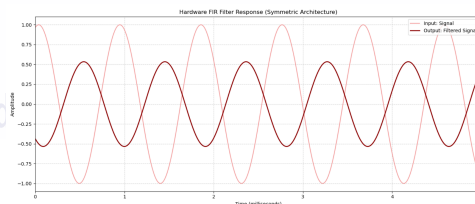


Fig. 0.2: 1100Hz

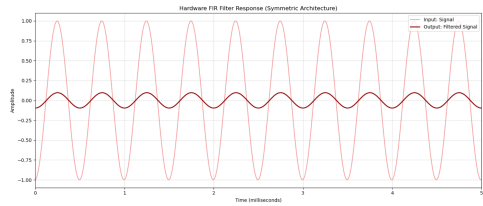


Fig. 0.3: 2000Hz

4 FIR Pipelined

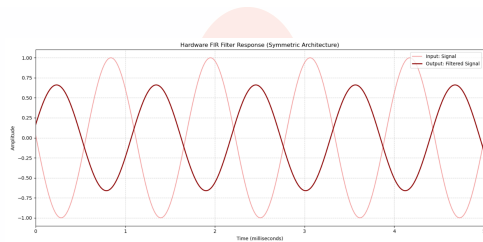


Fig. 0.4: 900Hz

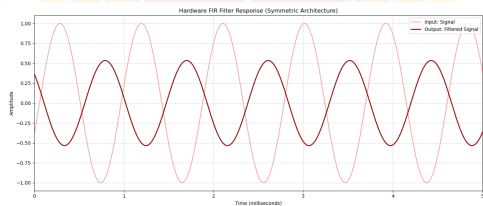


Fig. 0.5: 1100Hz

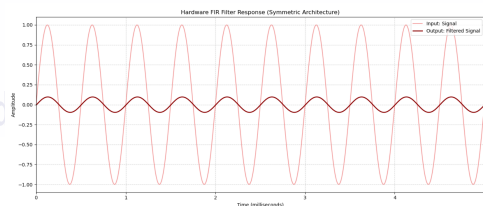


Fig. 0.6: 2000Hz

log: Trg @ 2026/04/26 23:03:16 (0-0.0.0: elapsed)			click to reset time bar																																
Type	Area	Name	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
		± input_signal																																	
		± m_imag																																	
		real																																	
		± r2sdf-u, m3fft, real10_012																																	
		m3_done																																	